INFORME FINAL

AgroSierra
AgroAPI + AgroCampo

Implementación, pruebas y evidencias · Examen Parcial II · Unidad 2 · Versión A

ESTUDIANTE	FECHA
Alexander Guillen Mendoza	27 de octubre de 2026

Repositorios publicados

Backend: github.com/alexgm-A11/AgroSierra-AgroAPI
Aplicación: github.com/alexgm-A11/AgroSierra-AgroCampo
Etiqueta publicada: **v1.0-unidad2** en ambos repositorios.

Alcance

Sistema independiente para AgroSierra, separado de PharmaMobile y de los ejercicios de biblioteca. Incluye una API protegida con JWT, Oracle ejecutado en Docker y cliente móvil Android con caché local y sincronización. Se excluye la Parte II de cambio individual porque así se indicó.

Resumen

La entrega reúne arquitectura por capas, reglas RN-01 a RN-04, matriz de permisos, datos de demostración, pruebas de backend y Android, enlaces Git y capturas reales del emulador Pixel 6. No se afirma verificación iOS: en este entorno se probó Android.

1. Solución y arquitectura

La implementación se separa en dos repositorios y mantiene la dirección de dependencias hacia el dominio. AgroAPI resuelve autenticación, autorización, reglas y persistencia; AgroCampo implementa la interfaz Android, estado de presentación, casos de uso compartidos y almacenamiento/sincronización local.

Backend · AgroAPI

Java y Spring Boot con Spring Security, JWT, JPA, Oracle Free y Flyway. Docker Compose levanta Oracle en el puerto local 1523 y la API en el 8080. Las migraciones crean esquema, matriz de privilegios y datos de demostración.

Capas y paquetes

Capa	Responsabilidad	Componentes
Web / DTO	Contrato HTTP versionado; valida solicitudes y responde modelos de transporte.	Controladores, DTO y errores uniformes.
Aplicación	Coordina casos de uso, permisos y reglas.	Servicios de autenticación, lotes, labores y cierre.
Dominio	Entidades y reglas del negocio agrícola.	Lote, Labor, TipoLabor, usuarios, roles y privilegios.
Datos	Persistencia y evolución del esquema.	Repositorios JPA, Oracle, migraciones Flyway y semilla.
Seguridad	Identidad y autorización basada en privilegios.	Filtro JWT, usuario actual y matriz de permisos.

Cliente Android · AgroCampo

Kotlin Multiplatform con Compose. En shared, domain contiene modelos, contratos y casos de uso; data contiene DTO, Ktor, repositorios, SQLDelight y sincronización offline-first; presentation contiene ViewModels y estado observable. androidApp aporta punto de entrada y adaptadores nativos. La sesión se almacena de forma segura.

Flujo de datos

Compose → ViewModel → caso de uso → repositorio → API/Ktor o SQLDelight → mapeo al dominio → estado observable → interfaz. En línea se envían operaciones pendientes y se refrescan lotes/labores; sin conexión, los cambios quedan locales para reintento.

2. Funcionalidad, seguridad y reglas

Las rutas se versionan bajo /api/v1. Todas requieren Bearer JWT salvo login. Las contraseñas demo se almacenan con BCrypt. El token expira a los 60 minutos e incluye identidad, rol y privilegios; cada petición carga al usuario desde la base para respetar permisos vigentes.

Endpoints principales

Método y ruta	Uso
POST /auth/login	Autentica y emite JWT.
GET /usuarios/me	Devuelve identidad, rol y privilegios vigentes.
GET /lotes	Paginación y filtros por valle y estado de campaña.
GET /lotes/{id}; GET /lotes/{id}/labores	Detalle de lote y labores paginadas.
POST/PUT/DELETE /labores	Crear, actualizar y eliminar labores con autorización.
POST /lotes/{id}/cerrar-campania	Cierre de campaña sujeto a privilegio y cosecha.

Reglas de negocio

RN	Regla	Resultado al incumplir
01	No crear ni modificar labores de campaña cerrada.	409 Conflict
02	Técnico edita solo su labor dentro de las primeras 24 horas.	403 Forbidden
03	Cerrar campaña exige al menos una labor de Cosecha.	409 Conflict
04	Fumigación ≤ dosis máxima por hectárea × hectáreas del lote.	400; validationErrors.cantidad

Datos iniciales

8 lotes distribuidos en tres valles, 20 labores y usuarios TECNICO, JEFE_CAMPO y ADMINISTRADOR. Dos lotes están cerrados; hay casos abiertos con y sin cosecha para probar RN-03. La colección Postman está en AgroAPI/postman.

3. Pruebas y resultados

Las verificaciones se realizaron con Docker iniciado, Oracle saludable y servicio backend activo. También se ejecutaron pruebas comunes y se instaló la app en un emulador Pixel 6 conectado al backend.

Ámbito	Comando o escenario	Resultado
Backend	mvnw.cmd -q test	EXIT 0; pruebas de servicios/reglas pasan.
Android	gradlew.bat :shared:testAndroidHostTest :androidApp:assembleDebug	BUILD SUCCESSFUL; APK Debug generado.
Docker	docker compose ps	API Up; Oracle Up (healthy).
Login	Credenciales válidas / inválidas	200 / 401
Seguridad	Sin JWT / falta de privilegio	401 / 403
RN-01	Crear labor en campaña cerrada	409
RN-03	Cerrar lote sin cosecha	409
RN-04	Fumigación sobre máximo	400; campo cantidad informado
RN-02	Técnico modifica labor ajena / propia	403 / 200
Labor CRUD	Registro / edición / borrado	201 / 200 / 204
Integración	Login Android y sincronización real	20 labores recibidas

Pruebas comunes y ejecución móvil

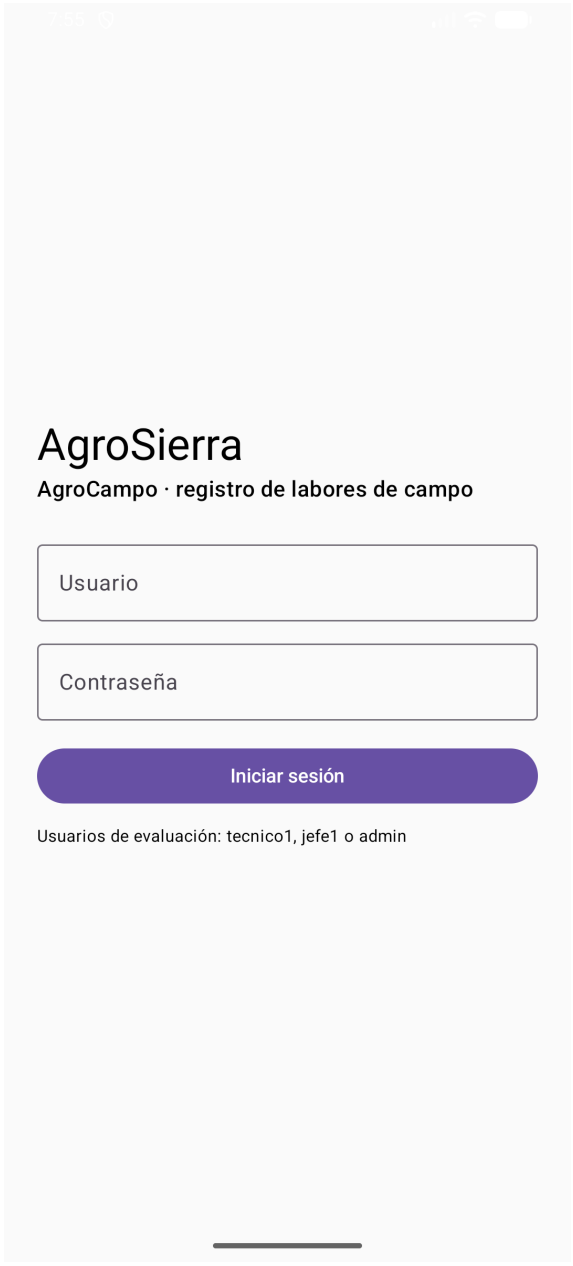
Se ejecutaron los casos comunes de mapeo DTO, traducción de errores HTTP, registro offline y orden de sincronización. La compilación generó androidApp/build/outputs/apk/debug/androidApp-debug.apk. En el emulador se comprobó login de TECNICO y actualización del padrón; la pantalla informó 0 enviados y 20 recibidos.

Observación de build

Maven muestra una advertencia informativa de Mockito sobre auto-adjunción del agente JVM; terminó con código 0 y no impidió las pruebas.

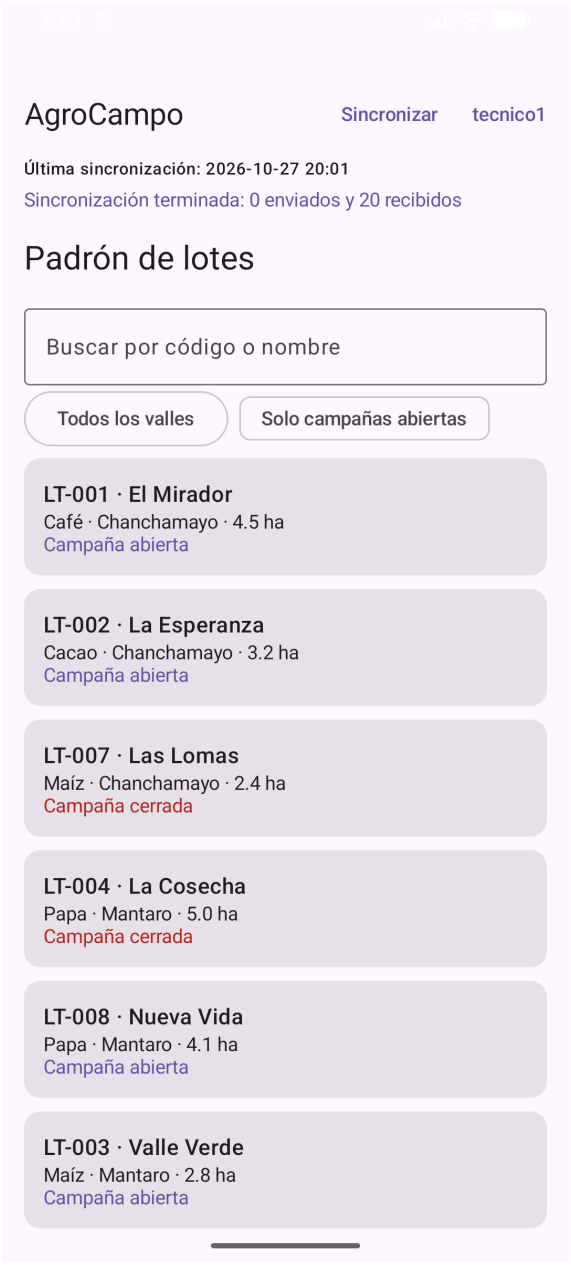
4. Evidencias Android

Capturas reales del emulador Pixel 6 conectado a AgroAPI en Docker.



1. Inicio de sesión

Formulario de acceso a AgroCampo en el emulador.



2. Padrón sincronizado

Sesión válida; sincronización terminada y 20 labores recibidas.

4. Evidencias Android (continuación)

Capturas reales del emulador Pixel 6 conectado a AgroAPI en Docker.



3. Listado de lotes

Lotes abiertos/cerrados con valle, cultivo y superficie.

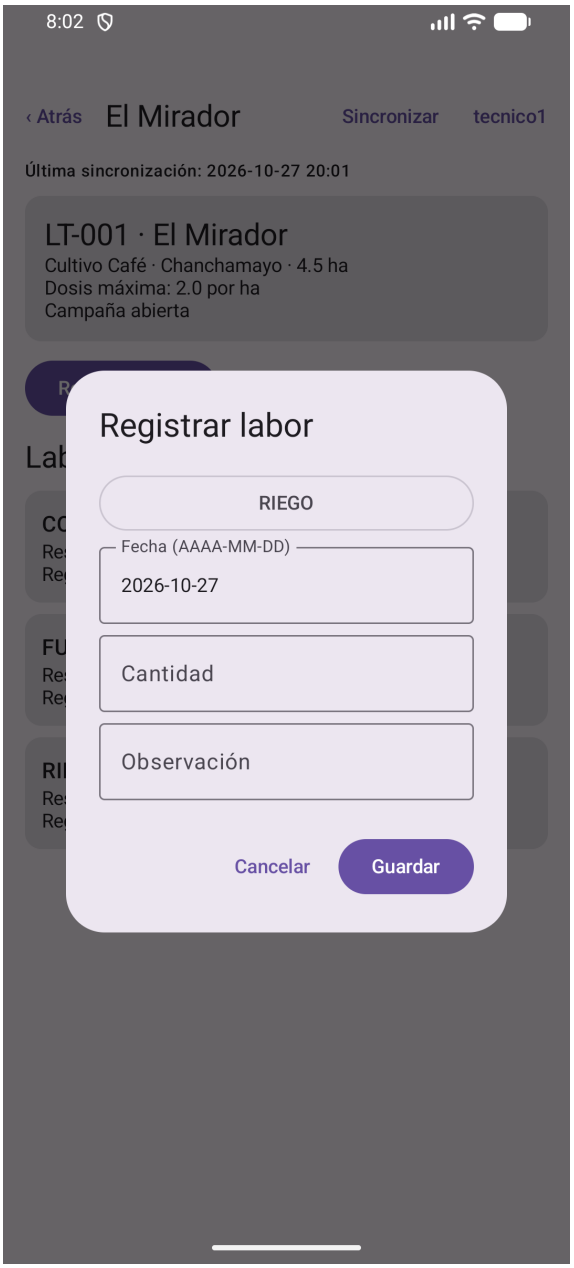


4. Detalle del lote

El Mirador, campaña abierta y labores existentes.

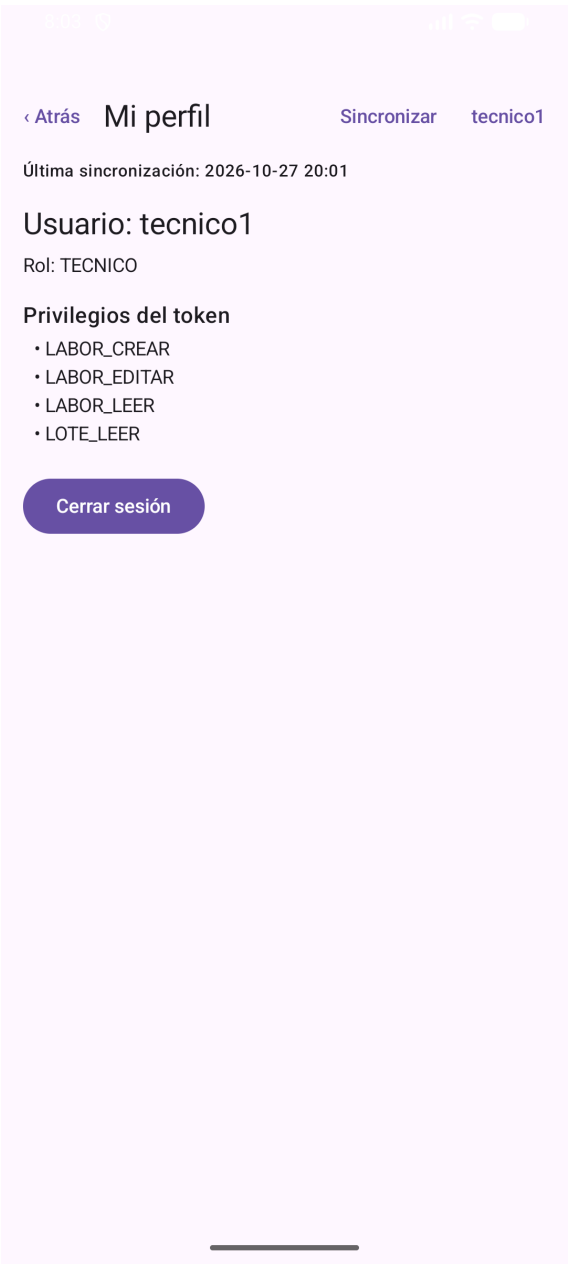
4. Evidencias Android (continuación)

Capturas reales del emulador Pixel 6 conectado a AgroAPI en Docker.



5. Registro de labor

Tipo, fecha, cantidad y observación.



6. Perfil y permisos

Identidad TECNICO y privilegios de la sesión.

5. Repositorios, ramas y alcance

Repositorios públicos en alexgm-A11

AgroAPI: github.com/alexgm-A11/AgroSierra-AgroAPI
AgroCampo: github.com/alexgm-A11/AgroSierra-AgroCampo
Etiqueta de entrega: v1.0-unidad2 en ambos repositorios.

Repo	Ramas de funcionalidad/corrección	Integración
AgroAPI	feature/modelo-y-permisos feature/api-reglas-jwt feature/pruebas-semillado fix/oracle-number-types	develop → main tag v1.0-unidad2
AgroCampo	feature/dominio-offline-first feature/api-sesion-segura feature/interfaz-pruebas	develop → main tag v1.0-unidad2

Historial

git shortlog -sne --all del repositorio Android registra 10 commits de Alexander Guillen. README de ambos repositorios explica estructura, decisiones de arquitectura, credenciales de demostración y ejecución. AgroAPI contiene la colección Postman; AgroCampo incluye pruebas comunes y capturas del emulador.

Parte II omitida

La Parte II de cambio individual se dejó fuera por instrucción expresa. Por ello, no se creó una rama SC ni se atribuye una solicitud de esa parte.

iOS

La ejecución y evidencias de este informe son Android (Pixel 6 API 37.1). No se generó compilación o captura iOS en este entorno; el informe no declara ese requisito como verificado. Se requiere macOS/Xcode y simulador iOS para completar esa evidencia.

Conclusión

AgroSierra quedó implementado y verificado en el flujo Android/API con Oracle en Docker. Reglas de negocio, autorización, persistencia y sincronización desde Android se comprobaron con pruebas automatizadas y escenarios HTTP. El pendiente de la entrega es la evidencia iOS.